Repository: writerish07-pixel/euler-crm · Commit 5098ee8 · Generated 16 Oct 2026

Euler CRM — Full Code Audit (2026-10-16)

Scope: the whole repository on `main` @ `5098ee8`. That covers the FastAPI backend (`backend/`, about 30k lines with `server.py` at 17k), the React app (`frontend/`), the Cloudflare workers, deploy config, and the legacy `assets/js` Apps Script portal.

How this was checked

Check	Result
`ruff` (pyflakes and bugbear) on backend	No undefined names, no syntax errors. 2 dead locals.
ESLint (undefined vars, hooks rules) on `frontend/src`	0 errors, 10 `exhaustive-deps` warnings
`CI=true react-scripts build`	**Compiles cleanly**
Frontend tests (`react-scripts test`)	**31 suites / 130 tests pass**
Backend tests (mongomock, offline)	**1277 passed, 59 failed, 17 errors**. 11 more files cannot run at all (see §5).
Endpoint cross-check: every literal API call in `frontend/src` vs the 255 backend routes (method and path)	**All 266 calls resolve to a real route.** 1 duplicate backend route (see §4).
Page guard vs API permission cross-check	No page whose own API always 403s for the roles that can open it
Live reproductions	Each item marked REPRODUCED was run against the real app with httpx and mongomock

Severity: **Critical** means money or customer data is wrong or exposed today. **High** means it will corrupt data or expose data under normal use. **Medium** is a real defect with a narrower trigger. **Low** is a robustness or hygiene issue.

---

1. Critical

C1. Refunds on multi-vehicle orders use only vehicle 1's price, so money the customer still owes can be refunded REPRODUCED

- `backend/server.py:9576` `_refund_position()` computes payable with `ce.compute_commercial_totals(lead_to_snapshot(lead))`. For a same-order pack, that snapshot covers **unit 1 only**. The code says so itself at `server.py:6985` (`_receipt_payable`: *"Payment posting used to fall through to compute_commercial_totals on the lead snapshot (unit 1)"*). `recompute_lead()` and `_receipt_payable()` were fixed to sum the units. `_refund_position()` was not.
- Reproduction: a 2-unit Turbo Max order has payable ₹15,94,000 and ₹11,09,500 received, so ₹4,84,500 is still owed. `GET /refund-position` reports `customerPayable 809500`, `excessReceived 300000`. `POST /refund` of ₹1,00,000 then **succeeds**, and outstanding rises to ₹5,84,500.
- The same unit-1 payable is used by `_rebuild_payment_running_totals()` (`server.py:9461`). After any receipt is deleted on a pack, every receipt's `outstandingBalance` is rewritten wrong, and the Payment Ledger sheet with it.
- Fix: use `_receipt_payable(lead, scheme_rows)`, or the persisted `lead["customerPayable"]` that `recompute_lead` already wrote, in both functions.

C2. The go-live reset restarts lead numbering but keeps lead-keyed data, so new customers inherit old customers' KYC and WhatsApp chats REPRODUCED

- `POST /api/admin/reset-transactions` (`server.py:16690`) sets `counters.lead = 0`. It does **not** clear `lead_documents`, `whatsapp_threads`, `whatsapp_messages`, `whatsapp_outbox`, `lead_requests` or `deal_verify_calls`.
- Reproduction: lead LD26000011 "Old Customer" has an Aadhaar upload and a WhatsApp thread. After the reset, the 11th new lead is again `LD26000011` ("Brand New 10"), and `GET /leads/LD26000011/documents` returns the **old customer's Aadhaar**. The old WhatsApp thread attaches to the new customer.
- Same root cause, worse effect: claim IDs are `CLM-{leadId}-{component}` (`server.py:1453`). The reset deliberately preserves the **Scheme Claim Register** sheet as a "permanent ledger". The sheet upsert is keyed on `claimId` (`gsheets.py:1501`), so the first scheme claim on a reused lead ID **overwrites a historical (possibly Received) claim row**.

- Fix: never reuse IDs. Keep the counters, or use a new prefix or epoch after a reset. Also clear, or archive, every lead-keyed collection.

C3. A public, unauthenticated page lists every customer

- `GET /api/share/dashboard` (`server.py:16485`) is on the `public` router. It returns `allLeads`, meaning **every lead's customer name**, model, variant, status and date, plus recent bookings and retail with names. `/share` (`frontend/src/pages/Share.js:169`) renders all of it. `memory/test_credentials.md` claims this board has *"no customer or staff data"*, which is false.
- It also loads at most 5,000 leads (`to_list(5000)`), so its totals silently go wrong past that.
- Fix: drop names from the public payload (counts by model and date only), or put the board behind a share token.

C4. Every deployment ships known demo logins, and all passwords are kept in plain text

- `backend/auth.py:681` `seed_users()` runs on **every startup** in every environment. If they are missing, it creates `executive@`, `accounts@`, `asm@`, `rm@`, `salesgm@` and `oemfinance@euler.com` with password **`euler@123`**, plus the owner, whose default is also `euler@123` unless `OWNER_PASSWORD` is set. `accounts` is a money-desk role. These credentials are published in `memory/test_credentials.md`.
- Every user row stores `passwordPlain` (`auth.py:491, 533`, change-password, reset). `GET /api/auth/users` returns each user's plaintext password to the owner. `password_matches()` (`auth.py:100`) also accepts the stored plaintext and whitespace-trimmed variants.
- No login rate limiting. A failed lookup scans the whole `users` collection (`auth.py:229`). Changing a password does not invalidate existing 7-day JWTs.
- Fix: seed demo users only when `ENVIRONMENT != production`. Remove `passwordPlain` and offer reset-only. Add a per-IP/account throttle. Add `passwordChangedAt` and check it against the token's `iat`.

---

2. High

H1. Lead search returns HTTP 500 for phone numbers typed with "+" (and for "(" or "[")

REPRODUCED

- `server.py:4090-4092` puts the raw `q` into `$regex`. Typing `+919876…` raises *"nothing to repeat"*; `(abc` and `[x` crash the same way. `Leads.js:40` sends `q` on **every keystroke** with no debounce, so the register fails while the user is typing. It is also a ReDoS vector.
- Fix: `re.escape(q)` (as `auth.py` already does) and debounce the input.

H2. Executives can read and change leads assigned to other executives

REPRODUCED

`_require_lead_visible()` is applied to `/360` and `/documents` (both correctly 403), but not here. Reproduced as an executive on another executive's lead:

Endpoint	Result
`GET /leads/{id}/whatsapp` (`server.py:16067`)	200: full customer chat
`POST /leads/{id}/whatsapp/reply`, `/booking-confirm`, `/google-review` (`16073-16107`)	No scope check. Any logged-in role, including ASM/RM/accounts, can message any customer.
`POST /leads/{id}/units` (`4929`)	**200: adds a vehicle to someone else's deal. Customer payable changes.** The response includes mobile.
`POST /leads/{id}/activities` (`12176`)	200
`GET /leads/{id}/deal-sheet.pdf` (`11394`)	Passes auth (failed only on data validation)
`GET /leads/{id}/billing-summary` (`10987`), `/scheme-rules`, `/price-preview`	200 for any role
`POST /leads/{id}/convert-booking` (`7404`)	No ownership check

`team_lead` users are not team-scoped on `PUT /leads/{id}` or the deal-desk unit endpoints. `_visible_threads()` (`16130`) scopes only `executive`, so `team_lead`, `asm` and `rm` see every WhatsApp conversation.

H3. Changing a lead's variant leaves the per-unit record on the old vehicle, so schemes are computed for the wrong vehicle

REPRODUCED · failing test

- Owner `PUT /leads/{id}` changes Turbo Max **Maxx (PV) → City (FB)**. The lead gets `exShowroom 640000`, but `units[0]` stays `Maxx (PV) / 785000`. `GET /price-preview` and `GET /scheme-rules` both resolve **Maxx (PV)**, because they prefer

`units[idx]` (`server.py:7335`, `7844`). That flows into scheme allocation, OEM claims and the deal sheet.
- Failing test: `tests/test_scheme_retained_and_step_locks.py::test_owner_model_change_refreshes_exshowroom_and_realigns_scheme`.

H4. Unauthenticated webhooks can write data when no secret is configured

- `POST /api/integrations/botspace/webhook` (`server.py:16231`) checks the secret only if one is set, and the default is empty (`botspace.py:233`). Anyone can then post an inbound **"STOP"** for any customer's phone (`botspace.py:1548` sets `whatsappOptOut`), inject fake inbound messages into the inbox, or use the status branch as a phone → `leadId` lookup (`botspace.py:1525`).
- Exotel `/voice`, `/gather`, `/recorded`, `/status` (`16382-16421`) behave the same until the Exotel settings are first saved; a secret is auto-generated only on save (`exotel.py:232`). `GET /integrations/exotel/gather?leadId=LD26…&Digits=500000` writes `dealVerifyAmount` and the mismatch flag on any lead (`exotel.py:667`). It also accepts an attacker-chosen `RecordingUrl`, which is rendered as `<audio src>`.
- Secrets travel in the query string (`?secret=` / `?token=`) and end up in access logs, and they are compared with `!=` rather than `hmac.compare_digest`.
- Fix: fail closed when no secret is set, and require a header secret.

H5. Server dates are UTC but the business runs on IST

- `server.py:237` `today()` / `this_month()` and `period.py:27` are UTC. `botspace.py:74` / `exotel.py:83` and the frontend (`format.js:151`) use local IST. Every night from 00:00 to 05:30 IST the server's "today" is still yesterday:
- default payment, booking and activity dates are a day early, and activity `time` is stored as UTC `HH:MM`;
- on the **1st of the month** before 05:30, `_lead_price_as_of()` (`6431`) falls back to UTC today, so the **previous month's** Scheme Master / Turbo uplift applies, and new bookings land in the previous month's reports;
- "this month" and "today" dashboard tiles (including the public board) show the wrong period.
- Fix: one `today_ist()` helper used everywhere.

H6. Lists silently stop at fixed caps that disagree with each other

143 `to_list(N)` calls, with `N` ranging from 50 to 50000. Once data grows past a cap, records disappear from screens with no error, and screens stop agreeing: - `GET /finance` loads `db.finance.find().to_list(1000)` with **no sort** (`9828`). After 1,000 files, the newest ones vanish. `rebuild_finance_views` uses 2000. - `GET /payments` keeps the latest 2000 and **then** applies the month/search filter (`9417`). - `GET /bookings` caps at 1000 (`12130`). `GET /activities` caps at 2000 before scoping (`12164`). - `GET /leads` takes the newest 3000 **before** executive/team scoping and period filtering (`4074`). An executive whose leads are older than that sees an empty register. - Most reports use 5000 leads, while `/reports/monthly`-family code uses 20000. Totals will diverge. - Fix: push filters (period, scope) into the Mongo query and paginate. Never cap before filtering.

---

3. Medium

#	Issue	Where
M1	**Sales GM can post customer payments**, which contradicts `auth.py`'s own role doc (*"They do not post money"*) and a test that now fails (`test_sales_gm_dashboard.py::test_gm_is_deal_desk_not_money_desk`, gets 200). Both branches of the mode check are identical `SALES_ROLES + MONEY_ROLES`. Confirm the intended policy.	`server.py:9437-9449`
M2	Google Sheets formula injection: every write uses `valueInputOption="USER_ENTERED"`, so a customer name or narration starting with `=`, `+`, `-` or `@` runs as a formula in the owner's workbook. Mobile numbers and chassis numbers also get re-typed (leading `+`, `1E5`). Use `RAW`, or prefix `'`.	`gsheets.py:1469, 1480, 1539`
M3	Sheet row-number cache (`_idrow_cache`) has no TTL and is never re-verified. If a person sorts, inserts or deletes rows in the Sheet, later updates for record X are written onto whatever row now sits at X's old row number.	`gsheets.py:396, 1336`
M4	The frontend auto-retries non-idempotent POSTs (payments, refunds, bookings) on 502/503/504 and network errors, across two hosts (`withFallback`). The server's only duplicate guard is "same amount within 4 s", so a gateway 504 after a committed insert yields a duplicate receipt. Send an idempotency key.	`frontend/src/lib/api.js:268-279`, `server.py _add_payment_internal`
M5	Refund race: `_refund_position` → insert has no lock or transaction. Two concurrent refunds of different amounts can both pass the "excess" cap.	`server.py:9600-9640`

#	Issue	Where
M6	`GET /leads/{id}/360` **writes** to the database: it re-hydrates pack prices, runs `recompute_lead`, and can insert a booking-advance receipt (`_ensure_booking_advance_receipt`, `6342`) with a check-then-insert. Read-only roles (ASM/RM) trigger writes, and concurrent opens could double-post (not reproduced on mongomock).	`server.py:5762-5815`
M7	`CORS_ORIGINS` is listed as a config option (`_config_diagnostics`) but CORS is hard-coded to `allow_origins=["*"]`, so the setting is silently ignored.	`server.py:16581, 16728`
M8	Real-looking customer PII (names and 10-digit mobiles) is committed in `backend/data/euler_raw.json`, `_incoming/euler_raw.json` and `_incoming/Euler_Master.xlsx`. It is **seeded into any fresh database**, including production on first boot (`seed.py:397`).	repo data
M9	`test_iter32_lead_cancel.py::test_close_won_still_shows_on_oem_and_commercial_tabs` fails. A Pending scheme claim disappears from `GET /claims` once the lead is Close Won, although the endpoint docstring says persisted claims stay listed forever. Needs triage.	`server.py:14142`
M10	Seed counters are rebuilt whenever the `counters` collection is empty (`seed.py:425`), resetting `lead` to the sample max (10). After a restore without `counters`, new IDs collide with existing leads, and `leads.leadId` has **no unique index** (`server.py` startup indexes).	`seed.py:425`, `server.py:~17087`

4. Endpoint integrity ("broken endpoints")

- **Every frontend API call (266 literal call sites) maps to an existing backend route with the right method.** No 404-by-construction endpoints were found.
- **Duplicate route:** `GET /api/reports/scheme-allocation-impact` is defined twice (`server.py:13426` and `15737`). FastAPI serves only the first. The second (a different, richer response shape) is dead code. No frontend caller exists.
- Endpoints that return **500 instead of 422** on bad input:
- `GET /api/leads?q=` with regex metacharacters (H1);
- `POST /leads/{id}/payments` (and other `float` bodies) with `NaN` / `Infinity`, which Python's JSON parser accepts, raises `ValueError` / `OverflowError` in `round2`. Reproduced. No data was stored.
- `GET /api/reports/daily/executive/{name}`: any logged-in role can read any executive's MTD numbers (`server.py:3294`).

5. Test suite and CI (macro)

- **No backend CI.** The only workflow (`.github/workflows/cloudflare-validate.yml`) builds the frontend. The ~1,350 backend tests never run on PRs, which is how the regressions below got merged.
- **11 test files can never run offline.** They require a live server via `REACT_APP_BACKEND_URL` and crash at import (`test_auth_and_iter2`, `test_euler_crm`, `test_iter3/4/7/8/9/11/13_14/15/16_*`).
- **The 76 failures and errors break down as:**
- **Real regressions:** H3 (variant change), M1 (Sales GM can post money), M9 (Close Won claim).
- **Stale tests vs the newer pytest Google-write guard** (`tests/conftest.py`): `test_iter19_env_safety…` (12), `test_iter20_column_contract` (5; includes a real gap: Dealer Earnings field `oemUnpayableWriteOff` has no sheet column, so it is dropped on sync), `test_iter22_header_relocation` (2), `test_iter23_lead_register_fixed_header` (4).
- **Stale catalog data:** `test_iter27_price_master_reprice` (17 errors: variant `Hi-Load / TR-NC (HiLoad)` no longer exists), `test_iter33_model_ask` (model list changed), `test_oem_portal_claims` (404 vs 403).
- **Order-dependent** (pass alone, fail in the full run because modules share one mongomock DB): `test_scheme_allocation_engine`, `test_turbo_sept_exshowroom_uplift`, `test_iter29_daily_reports`, `test_iter22_reported_defects`.
- `pywebpush` → `http-ece` fails to build on Python 3.13. Production pins 3.12 via `backend/.python-version`, but a local or CI 3.13 install of `requirements.txt` fails.

6. Low / hygiene

- `server.py` is 17k lines with 143 ad-hoc caps. Several business rules are duplicated (payable computed in 4 places), which is the root cause of C1.
- No start command is committed for the backend (no Procfile or `railway.json`). Deploy depends on dashboard settings.

- `lead_docs.read_upload` reads the whole upload into memory before the size check. `save_doc` deletes the previous unique document **before** inserting the new one, so a failed insert loses both.
- `frontend/src/lib/api.js` `saveBlob` revokes the object URL right after `click()`, which can cancel downloads on Safari/Firefox.
- Service worker caches every same-origin `.png/.jpg` cache-first (`sw.js:47`). A replaced `scheme-circulars/2026-10.png` stays stale until `VERSION` is bumped.
- `oem_sync.py:1177` and `server.py:4548`: unused locals (`have`, `live`). Possibly unfinished logic.
- Legacy `index.html` + `assets/js` portal still points at a hard-coded Apps Script URL (`assets/js/config.js`), whose `.gs` source is no longer in the repo. It is dead weight and a confusion risk.
- 10 `react-hooks/exhaustive-deps` warnings (`LeadDrawer.js:1014/1032/1212/1218`, `NewLeadDrawer.js:77/109/141`, `OemFinance.js:39`, `TallyPackCheck.js:17`). These can leave stale unit/price state when the selected unit changes.

Suggested fix order

1. C1 (refund cap), C2 (ID reuse on reset), C3 (public PII), C4 (demo logins / plaintext passwords).
2. H1 (search 500), H2 (lead scoping), H3 (unit variant), H4 (webhook secrets).
3. Add a backend CI job, fix or retire the stale tests so the suite is green, and then tackle H5/H6.